

RESTAURACIÓN DE LA BASE DE DATOS NORTHWIND

Gestión y Manejo de Base de Datos II

Nombre: Raúl Heredia

Fecha: 19 de mayo de 2026

1. Introducción

En la presente práctica se realizó la restauración de la base de datos de ejemplo **Northwind** utilizando herramientas de virtualización mediante Docker y el sistema gestor de bases de datos MySQL.

El propósito principal de esta actividad fue comprender el proceso de creación, importación y manipulación de una base de datos relacional en un entorno moderno de desarrollo. Además, se buscó reforzar conocimientos relacionados con consultas SQL, relaciones entre tablas y modificación de registros.

La base de datos Northwind es ampliamente utilizada para fines académicos debido a que contiene información relacionada con productos, categorías, proveedores, pedidos y clientes, permitiendo practicar distintos tipos de consultas y operaciones de administración de bases de datos.

Durante el desarrollo de la práctica se levantó un contenedor Docker con MySQL, se creó la base de datos Northwind y posteriormente se importaron los archivos correspondientes a la estructura y a los datos. Finalmente, se realizaron diferentes consultas SQL para demostrar el correcto funcionamiento de la base de datos restaurada.

2. Restauración de la Base de Datos Northwind

Para iniciar la práctica se utilizó Docker con el objetivo de ejecutar MySQL dentro de un contenedor, facilitando así la administración del entorno de trabajo sin necesidad de instalar el servidor directamente en el sistema operativo.

Primero se verificó que el contenedor estuviera en ejecución mediante el siguiente comando:

```
docker start ucatec-mysql
```

Posteriormente se ingresó al entorno de MySQL utilizando:

```
docker exec -it ucatec-mysql mysql -uroot -p
```

Después de acceder al gestor de base de datos, se procedió a crear la base de datos Northwind con el siguiente comando:

```
CREATE DATABASE northwind;
```

Una vez creada la base de datos, se importaron los archivos:

- northwind-schema.sql
- northwind-data.sql

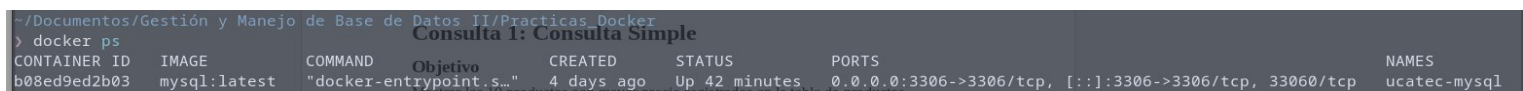
Estos archivos contienen la estructura de las tablas y los registros necesarios para el funcionamiento de la base de datos.

La importación permitió disponer de múltiples tablas relacionadas entre sí, como productos, categorías, proveedores, órdenes y clientes, entre otras.

Finalmente, se verificó que todas las tablas hayan sido creadas correctamente utilizando herramientas como Visual Studio Code.

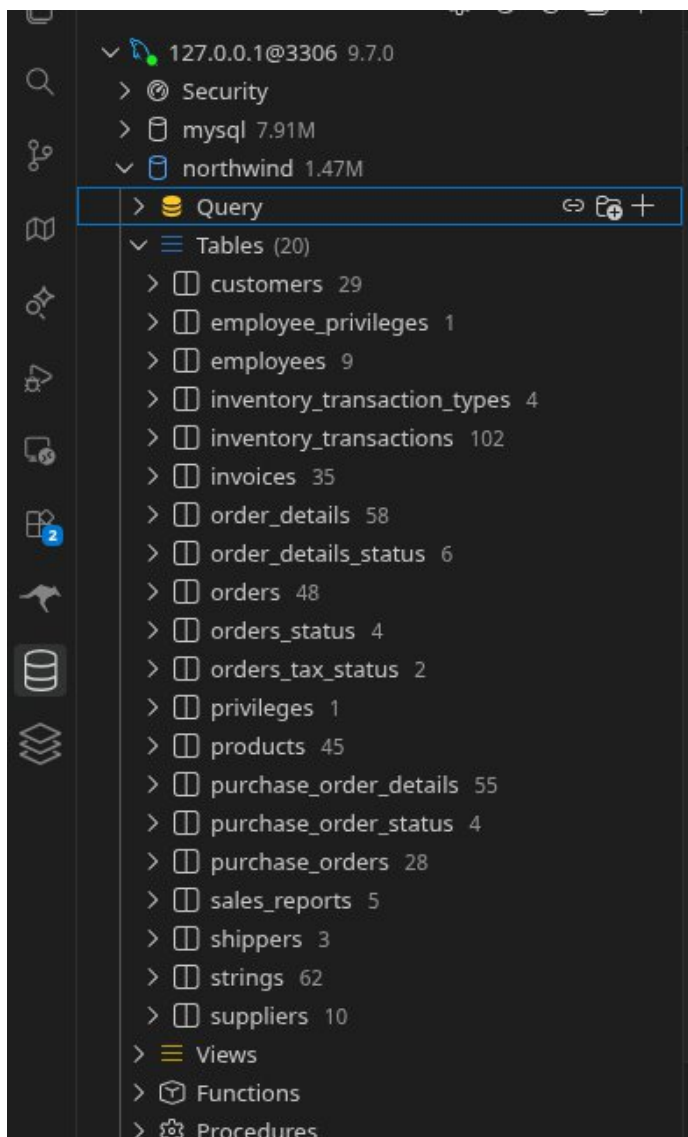
Capturas:

- Contenedor Docker ejecutándose correctamente (docker ps)



CONTAINER ID	IMAGE	COMMAND	Objetivo	CREATED	STATUS	PORTS	NAMES
b08ed9ed2b03	mysql:latest	"docker-entrypoint.s..."		4 days ago	Up 42 minutes	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp	ucatec-mysql

- Base de datos Northwind visible en VS Code



3. Consultas Realizadas

Una vez restaurada la base de datos, se realizaron diferentes consultas SQL para comprobar el correcto funcionamiento de las tablas y practicar distintos tipos de operaciones.

Consulta 1: Consulta Simple

Objetivo

Mostrar los 10 productos con mayor precio registrados en la tabla de productos.

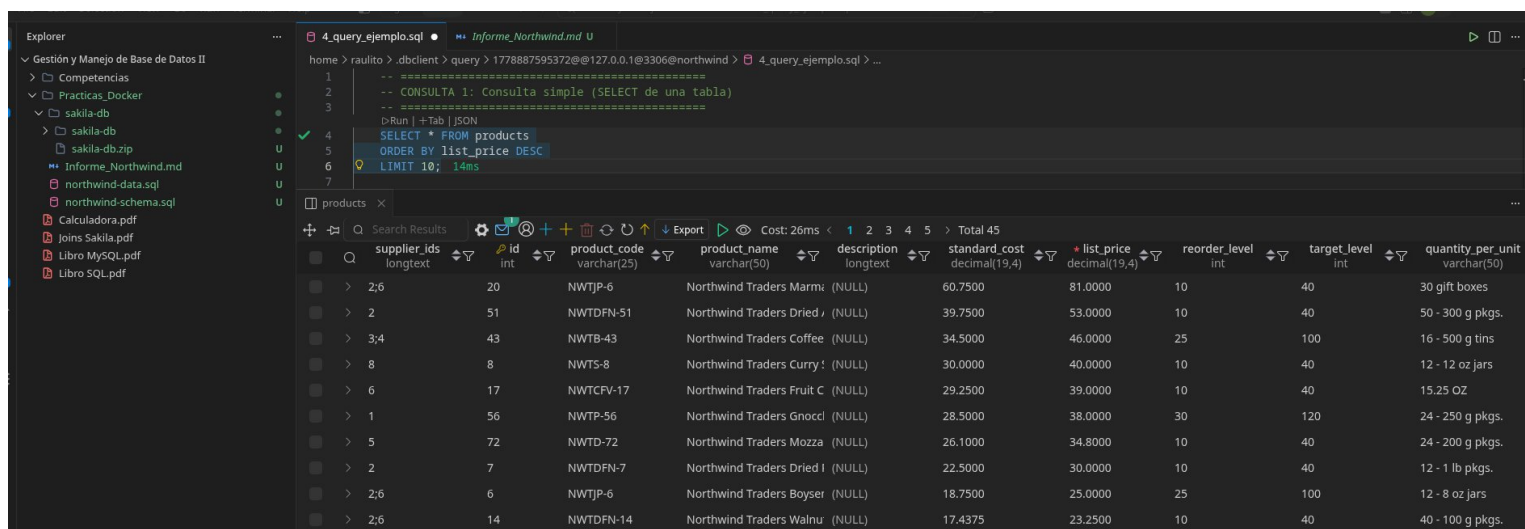
Consulta SQL

```
SELECT * FROM products
ORDER BY UnitPrice DESC
LIMIT 10;
```

Explicación

Esta consulta selecciona todos los campos de la tabla products, ordenando los productos de acuerdo con su precio de forma descendente. Posteriormente, mediante LIMIT 10, solo se muestran los diez productos más costosos.

La consulta permite identificar cuáles son los productos de mayor valor dentro de la base de datos.



The screenshot shows a SQL Server Enterprise Manager interface. On the left, the 'Explorer' pane shows the database structure. The central pane displays a query window with the following SQL code:

```
-- CONSULTA 1: Consulta simple (SELECT de una tabla)
-- =====
-- Run | +Tab | JSON
SELECT * FROM products
ORDER BY list_price DESC
LIMIT 10; 14ms
```

Below the query window, the 'Search Results' pane shows the results of the query. The results are displayed in a table with the following columns: supplier_id, id, product_code, product_name, description, standard_cost, list_price, reorder_level, target_level, and quantity_per_unit. The results are sorted by list_price in descending order.

supplier_id	id	product_code	product_name	description	standard_cost	list_price	reorder_level	target_level	quantity_per_unit
2:6	20	NWTJP-6	Northwind Traders Marmite	(NULL)	60.7500	81.0000	10	40	30 gift boxes
2	51	NWTFN-51	Northwind Traders Dried Fruit	(NULL)	39.7500	53.0000	10	40	50 - 300 g pkgs.
3:4	43	NWTB-43	Northwind Traders Coffee	(NULL)	34.5000	46.0000	25	100	16 - 500 g tins
8	8	NWTS-8	Northwind Traders Curry	(NULL)	30.0000	40.0000	10	40	12 - 12 oz jars
6	17	NWTCFV-17	Northwind Traders Fruit C	(NULL)	29.2500	39.0000	10	40	15.25 OZ
1	56	NWTP-56	Northwind Traders Gnocchi	(NULL)	28.5000	38.0000	30	120	24 - 250 g pkgs.
5	72	NWTD-72	Northwind Traders Mozzarella	(NULL)	26.1000	34.8000	10	40	24 - 200 g pkgs.
2	7	NWTFN-7	Northwind Traders Dried Fruit	(NULL)	22.5000	30.0000	10	40	12 - 1 lb pkgs.
2:6	6	NWTP-6	Northwind Traders Boyseberry	(NULL)	18.7500	25.0000	25	100	12 - 8 oz jars
2:6	14	NWTFN-14	Northwind Traders Walnuts	(NULL)	17.4375	23.2500	10	40	40 - 100 g pkgs.

Consulta 2: INNER JOIN

Objetivo

Mostrar información de los productos junto con la categoría a la que pertenecen.

Consulta SQL

```
SELECT
    p.ProductName,
    c.CategoryName,
```

```

        p.UnitPrice,
        p.UnitsInStock
FROM products p
INNER JOIN categories c ON p.CategoryID = c.CategoryID
ORDER BY p.UnitPrice DESC
LIMIT 15;

```

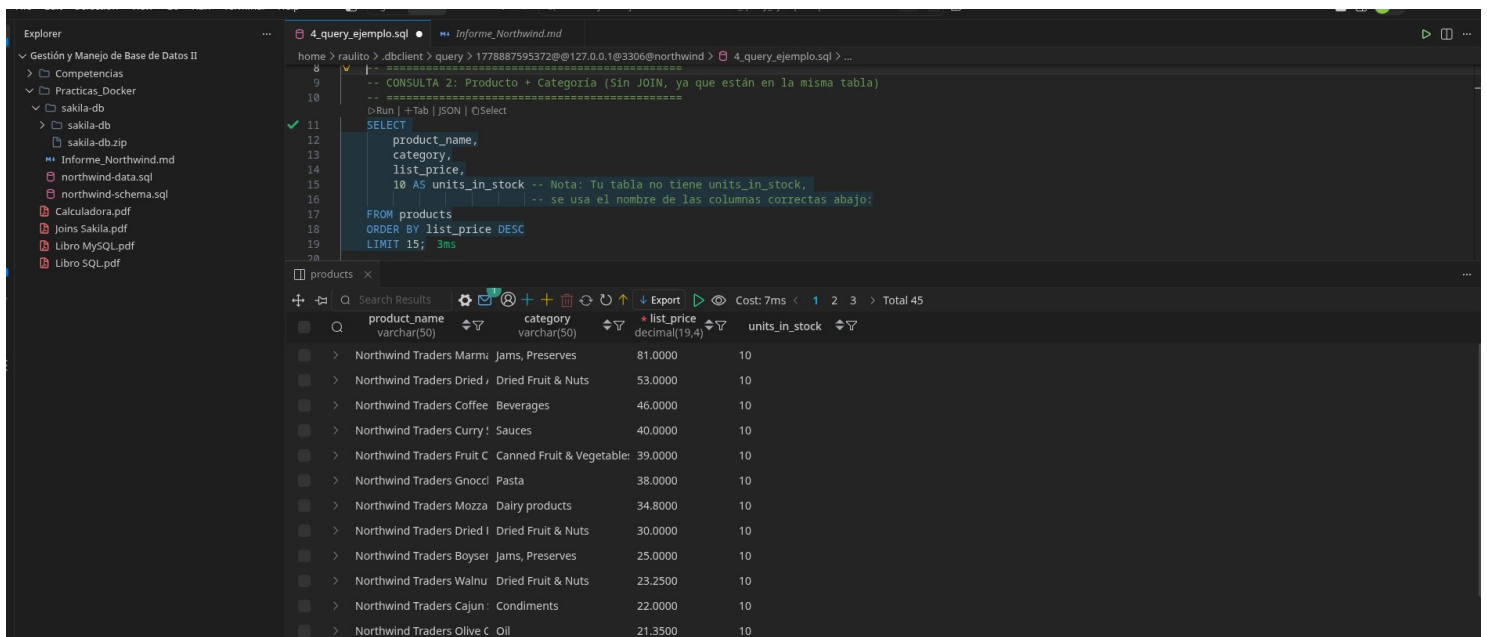
Explicación

En esta consulta se utilizó INNER JOIN para relacionar las tablas products y categories mediante el campo CategoryID.

El resultado muestra:

- Nombre del producto
- Categoría
- Precio
- Cantidad disponible en stock

La consulta permite observar cómo funcionan las relaciones entre tablas dentro de una base de datos relacional.



The screenshot shows a database client interface with a SQL query editor and a results pane. The query is as follows:

```

-- CONSULTA 2: Producto + Categoría (Sin JOIN, ya que están en la misma tabla)
-- =====
SELECT
    product_name,
    category,
    list_price,
    10 AS units_in_stock -- Nota: Tu tabla no tiene units_in_stock,
                        -- se usa el nombre de las columnas correctas abajo:
FROM products
ORDER BY list_price DESC
LIMIT 15; 3ms

```

The results pane displays the following data:

product_name	category	list_price	units_in_stock
Northwind Traders Marm...	Jams, Preserves	81.0000	10
Northwind Traders Dried...	Dried Fruit & Nuts	53.0000	10
Northwind Traders Coffee	Beverages	46.0000	10
Northwind Traders Curry	Sauces	40.0000	10
Northwind Traders Fruit C...	Canned Fruit & Vegetable...	39.0000	10
Northwind Traders Gnocci	Pasta	38.0000	10
Northwind Traders Mozza	Dairy products	34.8000	10
Northwind Traders Dried I...	Dried Fruit & Nuts	30.0000	10
Northwind Traders Boyser	Jams, Preserves	25.0000	10
Northwind Traders Walnu	Dried Fruit & Nuts	23.2500	10
Northwind Traders Cajun	Condiments	22.0000	10
Northwind Traders Olive C...	Oil	21.3500	10

Consulta 3: Agrupación con GROUP BY

Objetivo

Mostrar la cantidad de productos y el precio promedio por categoría.

Consulta SQL

```
SELECT
    c.CategoryName,
    COUNT(p.ProductID) AS Cantidad_Productos,
    ROUND(AVG(p.UnitPrice), 2) AS Precio_Promedio
FROM categories c
INNER JOIN products p ON c.CategoryID = p.CategoryID
GROUP BY c.CategoryName
ORDER BY Cantidad_Productos DESC;
```

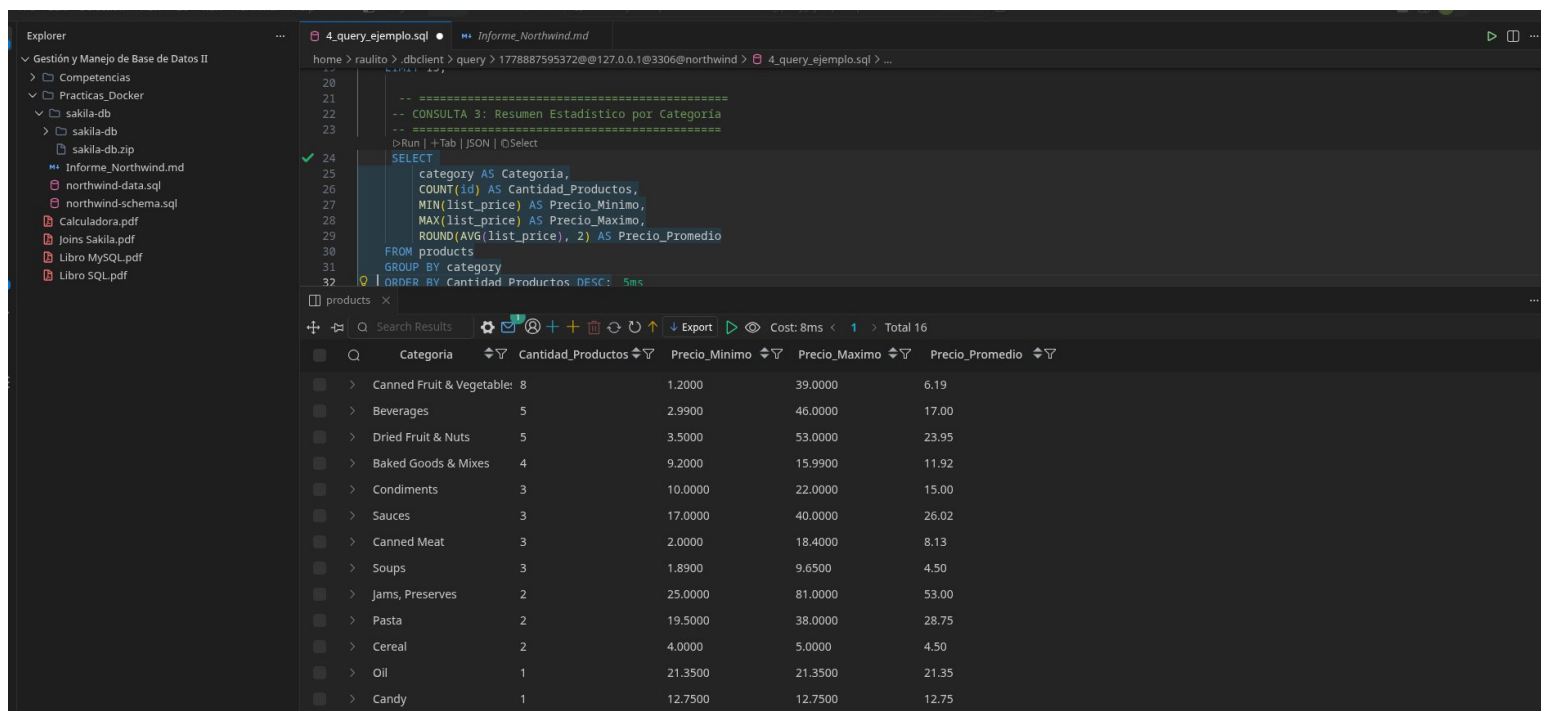
Explicación

En esta consulta se utilizó la función GROUP BY para agrupar los productos según su categoría.

Además:

- COUNT() permite contar la cantidad de productos por categoría.
- AVG() calcula el precio promedio.
- ROUND() limita el resultado a dos decimales.

Esta consulta ayuda a analizar estadísticamente la información almacenada en la base de datos.



The screenshot shows a SQL IDE interface. The left sidebar contains a file explorer with various files. The main editor displays a SQL query in a file named '4_query_ejemplo.sql'. The query is a SELECT statement that groups products by category, counting the number of products and calculating the average unit price, rounded to two decimal places. The results are ordered by the number of products in each category in descending order. Below the query editor, the 'products' table is selected, and the results of the query are displayed in a table format. The table has five columns: 'Categoría', 'Cantidad_Productos', 'Precio_Minimo', 'Precio_Maximo', and 'Precio_Promedio'. The results show 16 categories, with 'Canned Fruit & Vegetable' having the highest count of 8 products.

Categoría	Cantidad_Productos	Precio_Minimo	Precio_Maximo	Precio_Promedio
> Canned Fruit & Vegetable:	8	1.2000	39.0000	6.19
> Beverages	5	2.9900	46.0000	17.00
> Dried Fruit & Nuts	5	3.5000	53.0000	23.95
> Baked Goods & Mixes	4	9.2000	15.9900	11.92
> Condiments	3	10.0000	22.0000	15.00
> Sauces	3	17.0000	40.0000	26.02
> Canned Meat	3	2.0000	18.4000	8.13
> Soups	3	1.8900	9.6500	4.50
> Jams, Preserves	2	25.0000	81.0000	53.00
> Pasta	2	19.5000	38.0000	28.75
> Cereal	2	4.0000	5.0000	4.50
> Oil	1	21.3500	21.3500	21.35
> Candy	1	12.7500	12.7500	12.75

Consulta 4: INSERT y UPDATE

Objetivo

Insertar un nuevo producto y posteriormente actualizar su precio.

Consulta SQL

```
-- Insertar un nuevo producto
INSERT INTO products (ProductName, CategoryID, SupplierID, UnitPrice,
UnitsInStock)
VALUES ('Galletas de Chocolate Bolivianas', 1, 1, 12.50, 100);

-- Actualizar el precio del producto recién insertado
UPDATE products
SET UnitPrice = 11.99
WHERE ProductName = 'Galletas de Chocolate Bolivianas';

-- Ver el resultado
SELECT * FROM products
WHERE ProductName LIKE '%Galletas%';
```

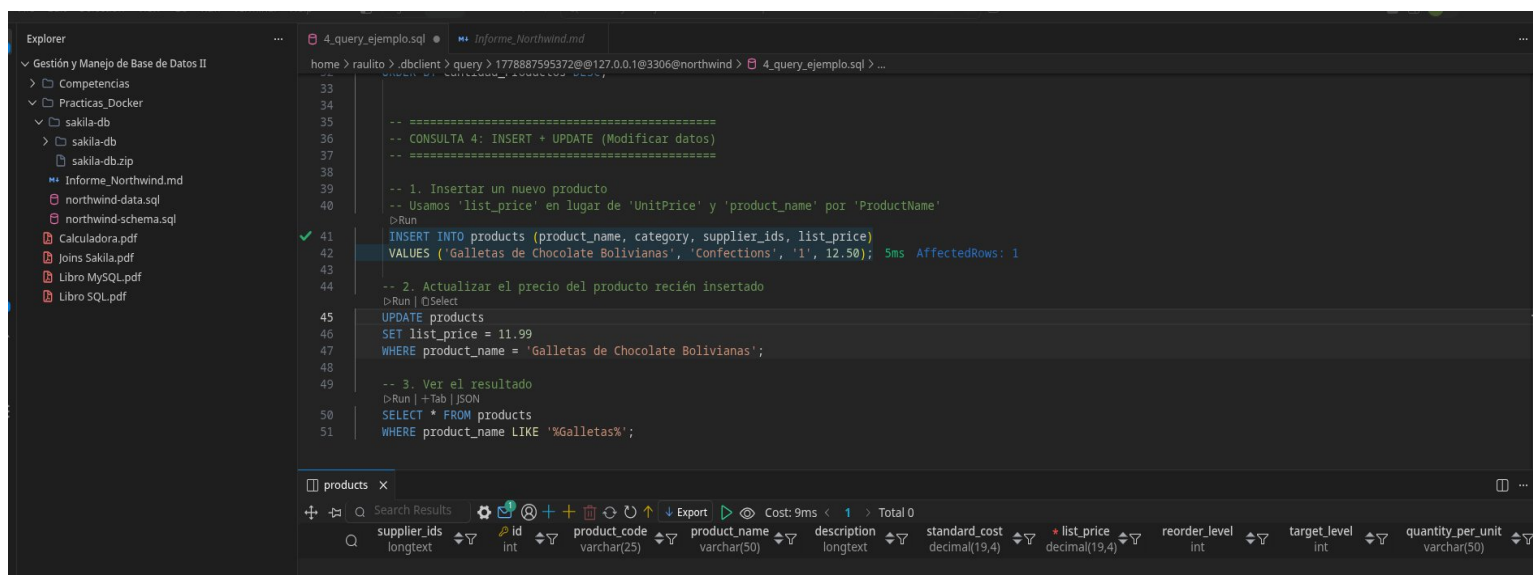
Explicación

Primero se insertó un nuevo producto dentro de la tabla products, asignándole nombre, categoría, proveedor, precio y stock.

Después se utilizó UPDATE para modificar el precio del producto insertado.

Finalmente, se ejecutó una consulta SELECT para verificar que los cambios se hayan realizado correctamente.

Esta práctica permitió trabajar con operaciones de manipulación de datos dentro de MySQL.



The screenshot shows a MySQL IDE interface. On the left, the 'Explorer' pane displays a file tree with folders like 'Gestión y Manejo de Base de Datos II', 'Practicas_Docker', and 'sakila-db'. The main editor area contains a SQL script with the following content:

```
33
34
35
36 -- CONSULTA 4: INSERT + UPDATE (Modificar datos)
37
38
39 -- 1. Insertar un nuevo producto
40 -- Usamos 'list_price' en lugar de 'UnitPrice' y 'product_name' por 'ProductName'
41 INSERT INTO products (product_name, category, supplier_ids, list_price)
42 VALUES ('Galletas de Chocolate Bolivianas', 'Confections', '1', 12.50); 5ms AffectedRows: 1
43
44 -- 2. Actualizar el precio del producto recién insertado
45 UPDATE products
46 SET list_price = 11.99
47 WHERE product_name = 'Galletas de Chocolate Bolivianas';
48
49 -- 3. Ver el resultado
50 SELECT * FROM products
51 WHERE product_name LIKE '%Galletas%';
```

Below the editor, the 'products' table is displayed with the following columns and data types:

supplier_ids	id	product_code	product_name	description	standard_cost	list_price	reorder_level	target_level	quantity_per_unit
longtext	int	varchar(25)	varchar(50)	longtext	decimal(19,4)	decimal(19,4)	int	int	varchar(50)

4. Conclusiones

- Se logró restaurar correctamente la base de datos Northwind utilizando Docker y MySQL.
- Se comprendió el proceso de creación e importación de bases de datos mediante archivos SQL.
- Se realizaron consultas simples, consultas con relaciones entre tablas y operaciones de modificación de datos.
- Se reforzaron conocimientos sobre el lenguaje SQL y el manejo de bases de datos relacionales.
- El uso de Docker facilitó la implementación de un entorno de trabajo rápido, ordenado y funcional para la práctica académica.